Mobile Robotics Lab

Week 02

Turtlesim Fundamentals

Submitted by: Fatima Mehboob Alam

2022-MC-34

Submitted to: Ma'am Maria Akram

Department of Mechatronics and Control Engineering

University of Engineering and Technology Lahore

Lab Report: Turtlesim Fundamentals

This report summarizes the practical exploration of the Robot Operating System 2 (ROS 2) using the `turtlesim` simulator. It details the steps taken to interact with, monitor, and manipulate nodes and topics.

1. Environment Setup and Basic Operation

The session began by initializing the ROS 2 environment and launching the core simulation components.

- **`ros2 run turtlesim turtlesim_node`**: Launched the graphical simulator window.
- **`ros2 run turtlesim turtle_teleop_key`**: Started the keyboard control node.
- **Observations**: We observed that the teleop node captures keystrokes and translates them into velocity commands for the default turtle (`turtle1`).

2. Topic Inspection and Communication

To understand the underlying "messages" being sent, we used several CLI tools:

- **`ros2 topic list`**: Identified active communication channels, specifically *`/turtle1/cmd_vel`* (velocity) and *`/turtle1/pose`* (position).
- **`ros2 topic echo`**: Monitored the *`/turtle1/cmd_vel`* topic in real-time, seeing the exact linear and angular values generated by the arrow keys.
- **`ros2 topic pub`**: Manually published a message to a topic. By sending a *`geometry_msgs/msg/Twist`* directly, we moved the turtle without using the keyboard.

3. Advanced Node Interaction (`rqt` and Services)

We used graphical tools and service calls to modify the simulation environment dynamically.

- **Using *`rqt`*:** Utilized `rqt_graph` to visualize the connection between the teleop node and the simulator node.
- **Service Commands**:

spawn: Added new turtles (e.g., `turtle2`, `turtle3`) at specific coordinates.
kill: Removed a specific turtle from the screen.
reset: Returned the simulator to its original startup state.
clear: Cleaned the "pen" lines left behind by the turtle.

- **Background Change**: Modified parameters to change the RGB values of the background color.

4. Advanced Control: Remapping

When multiple turtles were present, I discovered that the default teleop node only controls `turtle1`. To control **Turtle 3**, I used **remapping**:

- **Command**: *`ros2 run turtlesim turtle_teleop_key --ros-args --remap turtle1/cmd_vel:=turtle3/cmd_vel`*
- **Observation**: This redirects the output of the teleop node to a different topic without changing the underlying code, demonstrating the **modular nature** of ROS 2.

5. Final Observations

1. **Decoupling**: The *`turtlesim_node`* is "source agnostic"; it doesn't care if a command comes from a keyboard, a script, or a manual pub command—it only cares that the message arrives on the correct topic.
2. **Scalability**: By using remapping, we can control any number of independent robots using the same control logic.

Summary: The lab successfully demonstrated the ROS 2 "Publisher-Subscriber" model and showed how services allow for direct interaction with a robot's environment.